Comparison: bConnect-MCP vs mchluba/unofficial-baramundi-mcp-server

> Source: https://github.com/mchluba/unofficial-baramundi-mcp-server
> Compared against: this project (bConnect-MCP), as of 2026-04-01

High-Level Overview

Dimension	bConnect-MCP (this project)	mchluba/unofficial-baramundi-mcp-server
Scope	100% API coverage — 264 endpoints, 13 servers, ~253 tools	Use-case oriented — 15 tools covering the most common workflows
Architecture	13 domain-specific MCP servers (split)	Single monolithic MCP server
Transport	stdio (subprocess, Claude Desktop/Code native)	Streamable HTTP (remote-server model)
Authentication	HTTP Basic Auth (username + password)	Mutual TLS (mTLS with client certificates, Base64-encoded in `.env`)
Language	TypeScript	TypeScript
Runtime	Node.js	Node.js ≥ 22.14.0
API version	bConnect REST V2.0 (OpenAPI-backed, 25R2 + 26R1)	bConnect REST (version unspecified)
Tool philosophy		

Dimension	bConnect-MCP (this project)	mchluba/unofficial-baramundi-mcp-server
	API-mirroring (1 tool ≈ 1 endpoint)	Use-case-oriented (tools answer real IT admin questions)
Write-op guard	Not implemented	`ALLOW_WRITE_OPERATIONS=true` env var required
Startup validation	Not implemented	Verifies 6 API endpoints before accepting connections
Context efficiency	Engineered — load only the domain servers you need	N/A — single server, all 15 tools always loaded
Code generation	Yes — from OpenAPI specs (`src/generated/`)	Manual
Testing	vitest per server	Not mentioned
Docker	Not mentioned	Multi-stage Docker image (node:22-alpine)
License	—	Apache 2.0

Marco’s Design Philosophy: Use-Case Oriented Tools

The defining characteristic of mchluba’s server is the explicit choice **not** to mirror the API, but to design each tool around a real IT administrator intent. The README states:

> “use-case oriented MCP tools instead of a raw API mirror”

What this means in practice

API-mirroring style (like bConnect-MCP):

```
GET /endpoints → list_endpoints
GET /endpoints/{id} → get_endpoint
GET /updatemanagement/{id} → get_update_management_state
```

Use-case style (like mchluba):

```
find_windows_devices_with_missing_updates   → answers "which
devices need patches?"
find_devices_with_update_problems           → answers "which
```

```
devices have update issues?"
get_windows_device_details                    → answers "tell me
everything about this device"
```

The difference is that use-case tools often combine multiple API calls internally, returning a rich, pre-composed answer — rather than forcing the LLM to chain together several low-level tools.

Tool description pattern

Every tool description explicitly instructs the LLM on what to pass and what it gets back:

```
"Find Windows devices by registered user, last logged-on user,
host name, or display name.
Pass only the actual search value or fragment, not the full
natural-language question.
Use this when you need the devices for a person or when searching
for a laptop, PC, client,
or computer by name."
```

Input schema descriptions reinforce this:

```
        hostNames: z.array(z.string().min(1)).min(1).describe(
          "Pass exact Windows host names or exact display names only. Do
not pass fragments."
        )
```

This is deliberate guidance to prevent hallucinated inputs.

Marco's 15 Tools — Full Detail

Device Discovery

`find_windows_devices`

- **Input:** `query` — search value or fragment (name, user, display name)
- **Output:** list of unique Windows host names
- **Internal logic:** tokenizes query, removes punctuation, searches Active Directory, intersects results across terms, falls back to direct endpoint filtering by hostName/displayName/registeredUser/lastUser. Normalizes diacritics, handles German ß→ss, case-insensitive.

`get_windows_device_details`

- **Input:** `hostNameOrDisplayName` — fragment or exact

- **Output:** full device dossier with sections:
  - Overview (hostName, displayName, domain, OS, last contact)
  - Hardware (manufacturer, model, CPU, RAM)
  - Network (primary IP, primary MAC)
  - Variables (name, value, scope)
  - Windows Update (profile, missing critical/security/other counts, state)
- **Internal logic:** runs `Promise.allSettled()` across 3 API calls in parallel; partial failures are marked `status: "unavailable"` rather than failing the whole tool.

`get_windows_device_installed_software`

- **Input:** `hostNameOrDisplayName`
- **Output:** software inventory per matched device (name, vendor, version)

`get_windows_device_variables`

- **Input:** `hostNameOrDisplayName`
- **Output:** all endpoint variables for matched devices

`get_windows_device_jobs`

- **Input:** `hostNameOrDisplayName`
- **Output:** job assignments and execution states for matched devices

---

Update Management

`find_windows_devices_with_missing_updates`

- **Input:** none
- **Output:** all Windows devices with missing critical, security, or other updates; sorted by missing count descending (most critical first)
- **Use case:** “Show me all devices that need patches.”

`find_devices_with_update_problems`

- **Input:** `logicalGroup` — optional exact group name or group ID
- **Output:** devices with problematic update states, optionally scoped to a logical group
- **Defines “problem” as:** missing critical/security/other updates > 0, blocked updates > 0, deferred updates > 0, or state is “Unknown” / “InventoryOutdated” / “NonCompliant”

- **Severity scoring:** critical updates weighted ×1000 each for ranking
- **Use case:** “Which devices in the Finance group have update problems?”

Variables

`find_devices_by_variable`

- **Input:** `variableName` (exact), `value` (exact, optional)
- **Output:** devices where that endpoint variable is set, grouped by name/category/value
- **Use case:** “Find all devices where Location = Berlin.”

Job Management

`find_job_definitions`

- **Input:** `query` (string), `folderQuery` (optional)
- **Output:** matching job definitions with folder context
- **Use case:** “Find the patch deployment job.”

`find_job_definitions_by_folder`

- **Input:** `folderQuery` — exact folder name or full folder path
- **Output:** all job definitions in that folder and its subfolders
- **Use case:** “Show me all jobs in the Software Deployment folder.”

`get_job_instances_for_job_definition`

- **Input:** `jobDefinitionId` (exact)
- **Output:** all job instances for that definition (execution history)

`start_windows_job_definition_on_devices` ⚠️ Write

- **Input:** `jobDefinitionId`, `hostNames[]` (exact, array), `startIfAlreadyAssigned` (bool, default true)
- **Output:** per-device assignment result with status and job instance ID
- **Requires:** `ALLOW_WRITE_OPERATIONS=true`
- **Use case:** “Deploy the patch job to these 3 devices.”

`control_job_instance` ⚠️ Write

- **Input:** `jobInstanceId`, `action` (enum: `start` | `stop` | `resume`)

- **Output:** refreshed job instance state after the action
- **Requires:** `ALLOW_WRITE_OPERATIONS=true`

Software Inventory

`list_installed_windows_software_catalog`

- **Input:** none
- **Output:** aggregated software catalog across the entire environment, grouped by vendor and name
- **Use case:** “What software is installed across all our devices?”

`find_installed_windows_software`

- **Input:** `query` (string)
- **Output:** matching software groups with device counts, host names, and version history
- **Use case:** “Which devices have Adobe Reader installed?”

Notable Implementation Details in Marco’s Server

Startup Connectivity Check

Before accepting any MCP connection, the server calls 6 baramundi API endpoints to verify access (ActiveDirectory, Endpoints, Variables, Jobs, Software, UpdateManagement). If any fail, the server refuses to start:

```
        logger.info("baramundi.startup_check.started");
        await context.baramundiClient.verifyStartupAccess();
        logger.info("baramundi.startup_check.completed");
```

Write-Op Gate

All write tools check a config flag at registration time. `ALLOW_WRITE_OPERATIONS` defaults to `false` — write tools are blocked entirely unless the operator explicitly opts in:

```
        allowWriteOperations: parseBoolean(values.ALLOW_WRITE_OPERATIONS,
false),
```

Graceful Partial Failures

Device detail lookups use `Promise.allSettled()` so a failure in one sub-call (e.g. update status) does not fail the whole response. Each section is returned as `available` or `unavailable`.

Structured Result Rendering

Each search function has a dedicated `render*Result()` function that formats raw API data into readable LLM output with consistent section headers and field formatting helpers (`formatText()`, `formatNumber()`, `formatBoolean()`).

URL Normalization

A single `BARAMUNDI_SERVER_URL` is automatically split into 6 service-specific base URLs: `activedirectory`, `endpoints`, `variables`, `jobs`, `software`, `updatemanagement`.

Structured Logging

All tool calls emit structured log entries:

```
        context.logger.info("tool.find_windows_devices.started",
{ query });
        context.logger.info("tool.find_windows_devices.completed",
{ query, hostnames: result.hostnames });
```

Strengths of Each Approach

Marco's server excels at:

- **Usability out of the box** — 15 tools, one server, one config file
- **LLM friendliness** — tools answer questions, not expose endpoints
- **Safety** — write-op gate + mTLS + startup validation
- **Remote deployment** — Streamable HTTP + Docker = shared server for multiple users
- **Self-contained dossiers** — `get_windows_device_details` returns everything in one call

bConnect-MCP excels at:

- **Completeness** — 264 endpoints vs 15 tools; nothing in the bConnect API is out of reach

- **Version coverage** — explicit 25R2 and 26R1 tracking
- **Context efficiency** — load only the domain servers relevant to the current task
- **OpenAPI-backed** — generated from specs, less drift risk
- **Production hardening** — audit logging, batch ops, rate limiting per server
- **Domain separation** — 13 servers: assets, compliance, AD, defense control, update management, software, groups, jobs, endpoints, server management, and more

---

Things Worth Borrowing from Marco's Approach

Gap in bConnect-MCP	mchluba's solution	Effort
No write-op guard	`ALLOW_WRITE_OPERATIONS=true` env flag	Low
Basic Auth only	mTLS with Base64-encoded cert/key in env	Medium
No startup connectivity check	Verify 6 API endpoints before accepting connections	Low
Tools are API-shaped, not intent-shaped	Use-case-oriented tool naming and descriptions	High (design rethink)
No HTTP transport	Streamable HTTP for shared/remote deployment	Medium
No partial-failure handling	`Promise.allSettled()` in composite tool calls	Low

The highest-value, lowest-effort additions would be the **write-op gate** and **startup validation** — both are small additions with large safety and reliability payoffs.

Summary

Marco's server is **better for getting started and for real IT admin use cases** — it is opinionated, safe-by-default, and deploys cleanly as a shared remote server. An admin can connect it and immediately ask natural-language questions without knowing the bConnect API.

bConnect-MCP is **better for completeness and power users** who need precise access to the full API surface, or for automated workflows that require specific endpoint control not covered by 15 high-level tools.

The two approaches are complementary: Marco’s server demonstrates what a great user experience looks like for the most common 80% of use cases. bConnect-MCP covers the remaining 20% — and everything else.